セルフオーダーシステム 制作④ Service 層への整理

注文処理を Controller から Service へ分離する

Visual Studio 2022 ／ .NET 8　　Version 1.2

今回の制作で行うこと

制作③で完成した注文確定の機能はそのままに、長くなった OrdersController の処理を整理します。画面の動作を変えずに、コードの構造を改善する作業を「リファクタリング」といいます。

現在の構成	今回の構成
OrdersController が画面・検証・DB 保存を担当	OrdersController＝画面と HTTP の担当 OrderService＝商品検証と DB 保存の担当

Service 層とは？

「層（レイヤー）」とは、プログラムの仕事を役割別に整理したまとまりです。Service 層は「注文できる商品か確認する」「金額を確かめる」「注文データを保存する」など、注文の業務処理を担当します。今回は Services フォルダーに C#クラスを追加します。別のサーバーや DB を作るわけではありません。

役割	SelfOrderSystem の具体例
View（Razor）	画面に商品・ボタンを表示
Controller	リクエストを受け取り画面を切り替える
Service	売り切れ・価格を検査し注文を DB へ登録
Data／EF Core	AppDbContext を使って SQL Server を読み書き

今回の完成イメージ

ブラウザ（注文確定ボタン） → OrdersController（画面の受付）

→ IOrderService（利用するメソッドを定義） → OrderService（商品確認と注文保存）

→ AppDbContext（DB 操作） → SQL Server（データを保存）

到達目標

- Service 層の役割を Controller 層と区別できる
- IOrderService／OrderService を作り、DI を使って呼び出せる
- 画像・CSS・カート・注文確定・注文状況の動作を維持できる

学習の順序：STEP 1 変更前の保存 → STEP 2 interface と結果 → STEP 3 商品の検証 → STEP 4 注文保存 → STEP 5 DI 登録 → STEP 6 Controller 変更 → STEP 7 動作確認

STEP 1 変更前を Git に保存する

対象：Visual Studio 2022 の Git の変更／ブランチ

作業前の確認

- F5 で起動し、メニュー・写真表示・カート追加・注文確定・注文状況の動作を確認
- ［Git］→［Git の変更］を開き、変更ファイルと現在のブランチを確認
- 未コミットの変更があればコミットする（例：★制作 04 開始前 現状保存）
- 作業を分けたい場合は、現在の安定したブランチから「service-layer」を新規作成して切り替える

今回変更するファイル

場所	作業
Services/IOrderService.cs	新規作成
Services/PlaceOrderResult.cs	新規作成
Services/OrderService.cs	新規作成
Program.cs	登録 1 行を追加
Controllers/OrdersController.cs	コンストラクタと Place()を編集

変更しないもの

Product／Order／OrderItem、各 View（.cshtml）、site.css、写真ファイル、SQL Server の既存テーブルは変更しません。今回 Migration も不要です。

注意：Git で戻せるのは主にソース・画像などです。LocalDB 内の注文データはブランチを切り替えても自動で元に戻りません。

STEP 2 注文処理の interface と結果クラスを作る

新規作成：Services/IOrderService.cs と Services/PlaceOrderResult.cs

① IOrderService：利用できる注文処理を定義する

```
using SelfOrderSystem.Models;              // カートの商品情報の型を使う

namespace SelfOrderSystem.Services;

public interface IOrderService             // 注文処理に必要なメソッドを定義
{
    Task<PlaceOrderResult> PlaceAsync(     // 注文を確認して DB へ保存する
        List<CartItem> cartItems,          // Session から取得したカート
        int tableNumber);                  // 入力されたテーブル番号
}
```

② PlaceOrderResult：成功・失敗を Controller へ返す

```
namespace SelfOrderSystem.Services;

public class PlaceOrderResult              // 注文処理の結果をまとめる
{
    public Guid? PublicId { get; set; }    // 成功した注文の公開 ID
    public List<string> Errors { get; set; } = new(); // 失敗時のメッセージ
    public bool IsSuccess =>               // 成功かどうかを計算
        PublicId.HasValue && Errors.Count == 0;
}
```

ここで覚えること

- interface は、実装クラスが用意するメソッドの名前・引数・戻り値を定義する
- Task<PlaceOrderResult>は、非同期処理の完了後に PlaceOrderResult を受け取る型
- Guid?は null を許す Guid 型で、注文失敗時は PublicId に値を設定しない

STEP 3 OrderService に商品検証を移す（1/2）

新規作成：Services/OrderService.cs ｜ 続きは次ページに入力

STEP 3〜4 のコードはすべて 1 つの OrderService.cs に入力します。P4 ではクラスとメソッドを開いたままにし、P5・P6 を続けて入力します。P6 の最後まで入力する前はビルドできません。

```
using Microsoft.EntityFrameworkCore;      // EF Core の非同期検索
using SelfOrderSystem.Data;                // AppDbContext
using SelfOrderSystem.Models;              // Product と Order

namespace SelfOrderSystem.Services;

public class OrderService : IOrderService  // IOrderService で定義した処理を実装
{
    private readonly AppDbContext _db;     // DB 操作に使うフィールド
```

```
    public OrderService(AppDbContext db)    // DI から DbContext を受け取る
    {
        _db = db;                           // 受け取ったものを保持
    }

    public async Task<PlaceOrderResult> PlaceAsync(
        List<CartItem> cartItems, int tableNumber)
    {
        var result = new PlaceOrderResult(); // 注文結果とエラーをまとめる


        if (cartItems.Count == 0)           // カートが空なら中止
        {
            result.Errors.Add("カートが空です");
            return result;
        }


        if (tableNumber < 1 || tableNumber > 999) // テーブル番号の範囲を確認
        {
            result.Errors.Add("テーブル番号は 1〜999 にしてください");
            return result;
        }

        var ids = cartItems.Select(x => x.ProductId)
            .Distinct().ToList();          // 商品 ID の重複を除く


        var products = await _db.Products.AsNoTracking()
            .Where(p => ids.Contains(p.Id) && p.IsActive)
            .ToDictionaryAsync(p => p.Id); // 商品 ID をキーとする辞書で取得
```

カートの価格は「商品追加時」の情報です。注文確定までに売り切れや価格変更が起きるため、必ず現在の DB 情報を読み直します。

STEP 3 OrderService に商品検証を移す（2/2）

OrderService.cs の続き ｜ STEP 3 の ToDictionaryAsync 直後

```
        foreach (var item in cartItems)    // カートの商品を 1 件ずつ確認
        {
            if (!products.TryGetValue(item.ProductId, out var product))
            {
                result.Errors.Add("注文できない商品があります");
                continue;                  // 次の商品へ進む
            }


            if (product.IsSoldOut)         // 売り切れを確認
                result.Errors.Add($"{product.Name}は売り切れです");


            if (item.Quantity < 1 || item.Quantity > 99)
                result.Errors.Add("数量は 1〜99 にしてください");


            if (product.Price != item.UnitPrice) // 追加後の価格変更を確認
                result.Errors.Add($"{product.Name}の価格が変更されました");
        }


        if (result.Errors.Count > 0)       // エラーが 1 つでもあれば
            return result;                // 保存せずに結果を返す
```

処理の意味

コード	役割
TryGetValue	該当商品を DB 取得結果から探す
IsSoldOut	売り切れなら注文を止める
product.Price != item.UnitPrice	カート時点の価格から変更されたか調べる
result.Errors.Count > 0	保存前にエラーがあるか最終確認

商品が見つからない・売り切れ・数量不正・価格変更のいずれかがあれば、注文を保存せず失敗理由を返します。注文作成は次ページに続きます。

STEP 4 OrderService で注文を保存する

OrderService.cs の末尾 ｜ STEP 3 の if 文の直後に続ける

```
        var order = new Order             // 1 件の注文全体を作る
        {
            TableNumber = tableNumber,    // 画面から受け取った番号
            OrderedAtUtc = DateTimeOffset.UtcNow, // 注文時刻
            Status = OrderStatus.New       // 新規注文
        };

        foreach (var item in cartItems)   // 商品ごとに明細を作る
        {
            var product = products[item.ProductId]; // 確認済みの商品
            order.Items.Add(new OrderItem
            {
                ProductId = product.Id,   // 注文した商品 ID
                Quantity = item.Quantity, // 注文数
                UnitPrice = product.Price // DB で確認した単価
            });
        }

        _db.Orders.Add(order);             // 注文と明細を保存対象に
        await _db.SaveChangesAsync();      // SQL Server の保存完了を待つ

        result.PublicId = order.PublicId; // 完了画面で使う ID
        return result;                    // 成功結果を返す
    }
}
```

なぜここではカートを消さない？

Service は DB への注文保存まで担当します。Session の削除と完了画面への移動は Controller の仕事です。DB 保存に失敗したときはカートを消さないようにします。

補足：Order.PublicId には制作③で「= Guid.NewGuid()」を設定済みです。new Order の時点で生成されるため、再設定は不要です。

STEP 5 Program.cs に DI 登録する

既存編集：Program.cs ｜ var app = builder.Build(); より前

制作②の AddScoped<CartService>()は消さず、その近くに IOrderService の登録を 1 行追加します。既存の Program.cs 全体を置き換えないでください。

```
// Program.cs の先頭に using がなければ追加
using SelfOrderSystem.Services;

// builder.Services.AddScoped<CartService>(); の近くに追加
builder.Services.AddScoped<IOrderService, OrderService>();
```

この 1 行の意味

要素	役割
IOrderService	Controller が必要とする interface
OrderService	実際に注文を処理するクラス
AddScoped	1 回の HTTP リクエストを処理する間は同じ実体を使う登録

DI の流れ

① OrdersController が IOrderService を要求 → ② DI コンテナが OrderService を生成 → ③ OrderService に AppDbContext を渡す → ④ Controller から PlaceAsync()を実行する

チェック

- Program.cs を編集したらアプリを停止して再起動する
- IOrderService.cs、PlaceOrderResult.cs、OrderService.cs が Services フォルダーにある
- この時点で［ビルド］→［ソリューションのビルド］を実施する

STEP 6 OrdersController を整理する

既存編集：Controllers/OrdersController.cs

① コンストラクタを置き換える

既存の_db・_cart は他の Action でも使うため残します。_orderService を追加し、コンストラクタは元のものを丸ごと置き換えます。同じ引数構成のコンストラクタを重複させないでください。

```
private readonly IOrderService _orderService; // 注文の検証と保存を担当する Service


public OrdersController(AppDbContext db, CartService cart,
                        IOrderService orderService)
{
    _db = db;                               // 既存の DB 参照用
    _cart = cart;                           // 既存の Session 用
    _orderService = orderService;           // DI から受け取った Service
}
```

② Place()メソッドだけ丸ごと置き換える

```
[HttpPost]                                  // POST で注文を受け取る
[ValidateAntiForgeryToken]                  // 不正なフォーム送信を検証
public async Task<IActionResult> Place(CheckoutViewModel model)
{
    var cartItems = _cart.GetItems();       // Session 内のカートを取得
    if (!ModelState.IsValid)                // テーブル番号などが不正
    {
        model.Items = cartItems;           // 画面再表示のため戻す
        return View("Checkout", model);
    }

    var result = await _orderService.PlaceAsync(
        cartItems, model.TableNumber);      // 注文処理を Service へ依頼


    if (!result.IsSuccess)                  // 保存できない場合
    {
        foreach (var error in result.Errors)
            ModelState.AddModelError("", error); // エラーを画面へ
        model.Items = cartItems;
        return View("Checkout", model);
    }


    _cart.Clear();                          // 保存成功後だけカートを消す
    return RedirectToAction(nameof(Complete),
        new { id = result.PublicId });      // 完了画面へ移動
}
```

注意：Checkout()・Complete()・Status()・LoadPublicOrderAsync()は変更しません。注文確認・完了・状況画面の Razor もそのまま使います。

STEP 7 動作確認・Git コミット

確認対象：画面、Session、SQL Server、Git

正常系とエラー系を試す

操作	期待結果
メニューで商品を 2 種類追加	写真とカート表示は以前と同じ
注文確認から正しい番号で確定	注文完了・注文状況へ移動する
DB の Orders／OrderItems を確認	注文親 1 件・明細は種類数分が追加
テーブル番号 0 で注文	検証エラーが出て保存されない
商品を売り切れにして注文	保存されず、カートは残る
価格を変更して注文	価格変更エラー、カートは残る
注文成功後のカート	空になっている

困ったときは

症状	まず確認する場所
IOrderService が見つからない	ファイル名／namespace／using
Unable to resolve service	Program.cs の AddScoped 登録
OrderService のメソッドがない	PlaceAsync の綴り／引数／interface との一致
注文確定画面でエラー	Controller のモデルと View 表示
画面が変化しない	アプリの停止・再起動／実行中のブランチ

最後の Git 操作

- 差分で OrderService 関連と OrdersController の変更を確認する
- コミット例：★制作 04 Service 層へ注文処理を分離
- 作業ブランチを使った場合、採用する変更だけ main へマージする

まとめ：Controller は HTTP 要求と画面の制御、Service は注文の業務処理、AppDbContext は SQL Server へのデータアクセスを担当します。